

Physics-Informed Online Machine Learning and Predictive Control of Nonlinear Processes with Parameter Uncertainty

Yingzhe Zheng and Zhe Wu*



Cite This: *Ind. Eng. Chem. Res.* 2023, 62, 2804–2818



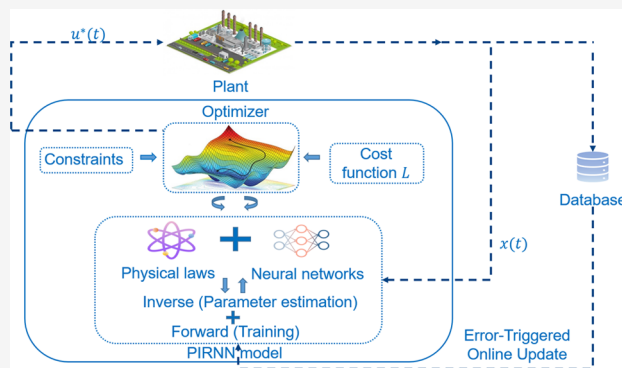
Read Online

ACCESS |

Metrics & More

Article Recommendations

ABSTRACT: In this work, we present a physics-informed recurrent neural network (PIRNN)-based modeling approach for nonlinear dynamic systems with parameter uncertainty. Physics-informed modeling approaches can improve the generalization performance of machine learning models by embedding the knowledge of physical laws in the learning process. Based on the standard PIRNN modeling approach for the nominal system without model uncertainties, we develop a novel PIRNN-enhanced modeling method that integrates online estimation of uncertain process parameters into the training process using the latest process data. The PIRNN-enhanced modeling approach is incorporated into the design of model predictive control (MPC), where an error-triggered mechanism is employed to trigger the quantification of modeling uncertainties and update of PIRNN models accordingly. Finally, a chemical process example is used to demonstrate the superiority of the proposed PIRNN-enhanced online learning mechanism in comparison to the conventional purely data-driven online update strategy for nonlinear systems subject to process parameter uncertainty.



INTRODUCTION

Model predictive control (MPC) is an optimization-based advanced control method that solves for the optimal control actions based on a process model while accounting for the intrinsic dynamic behaviors of the process. It is a highly flexible and powerful control scheme that permits the inclusion of practical constraints (e.g., temperature limits of a heating jacket) on the manipulated variables, thereby facilitating the search of feasible solutions to optimization-based control problems by respecting any physical limits imposed. Collectively, the convergence of next-generation information technologies and the pervasive nature of data in modern manufacturing complexes culminates in machine learning-based MPC (ML-MPC), which has attracted an increased level of attention in recent years, and is gaining traction in control of highly nonlinear processes.^{1–5} In ref 3 an ML-MPC is developed using an autoencoder-based recurrent neural network (RNN) to control the batch crystallization process, where the crystal size and yield are optimized. Additionally, the ML-MPC scheme is equipped with an error-triggered online update mechanism to mitigate issues pertaining to plant-model mismatch and to improve the overall control performance.³ A theoretical analysis in ref 6 further derives a generalization error bound for online learning-based RNN models. However, the development of an accurate surrogate RNN model is not a trivial task, often necessitating a comprehensive set of training data to warrant good modeling performance. The acquisition

of an exhaustive set of process data is thus deemed a major impediment to the implementation of ML-based MPC among many industry practitioners and researchers.

In view of the favorable results engendered by implementing regularization techniques in ML tasks, and in addition to constraining the magnitude of the network weights, it is proposed to supplement the training process by harnessing and embedding a priori mechanistic knowledge expressed in the form of ordinary differential equation (ODE) or partial differential equation (PDE) directly into the RNN loss function as soft penalty constraints, thereby making the learning algorithm physics-informed (PI).⁷ In addition to embedding physical laws in the loss function, physical knowledge can also be utilized to develop hybrid models and process-structure-based machine learning models.^{8–11} For example, in ref 11 sparse identification of nonlinear dynamics (SINDy) in discovering nonlinear dynamic is incorporated into machine learning models. In ref 10 process structural knowledge such as the relationships between the process input

Received: October 12, 2022

Revised: January 10, 2023

Accepted: January 11, 2023

Published: January 30, 2023



and output variables was used to design partially connected NN models to improve the generalization performance and reduce training time. The PI technique therefore effectively enforces the physical principles or the governing equations into the RNN architecture, and is uniquely advantageous in enhancing the trainability and generalizability of neural networks, especially in the small data regime, which is characterized by data scarcity or insufficiently sampled training data.^{12–14} This distinctive integration of expert domain knowledge in terms of the underlying physical laws and the process dynamics encapsulated in the available data informatively steers the training process toward the learning of mechanistically and observationally consistent solutions, and is applicable for solving both the forward and inverse problems. Recently, PI technology has been applied in various engineering problems (e.g., thermodynamics¹⁵ and fluid mechanics¹⁶). For example, in ref 17 a hybrid modeling strategy was developed to predict unknown process parameters in the hydraulic fracturing process.

In solving forward problems, it is stipulated that all process parameters are known and well-defined. However, complete knowledge of all of the process parameters is often arduous if not impossible to obtain due to the dynamic nature of the processes in the chemical industry. In real production systems, the process dynamics is constantly evolving and changing with respect to time, and these uncertainties in the system thus pose a significant hindrance and deter effective control by MPC. To address this issue, some recent works have developed MPC under uncertainty using ML methods.^{18,19} For example, in ref 20 the authors proposed a robust economic model predictive controller using the deep deterministic policy gradient framework under disturbances and parametric uncertainty. In addition to the design of robust controllers the quantification of model uncertainty and process disturbances resembles an inverse problem of PI learning, where the time-varying process parameters can be estimated based on the latest process data retrieved. Since the prediction performance of the PI-based ML model depends on the accuracy of the mechanistic model, accurate estimation of process parameters and hence prompt update of the PI-based ML model in response to the varying system dynamics are crucial to attain a more effective process control. It is therefore vital to incorporate real-time or online update of the process model within the MPC scheme to further enhance its performance. Previous works have utilized purely data-driven online model update strategies for RNN-based MPCs subject to time-varying disturbances, where real-time sensor data are collected and used to update the RNN model to improve its prediction accuracy, yielding improved control performance.²¹ In our previous work, a PIRNN model has been developed for the nominal system without model uncertainties;²² however, at this stage, PI-informed ML modeling of nonlinear systems with model uncertainties has not been studied. Therefore, in this work, we consider nonlinear systems subject to parameter uncertainty and further develop an inverse PIRNN modeling method to estimate unknown process parameters in order to improve the prediction accuracy of PIRNN models.

Motivated by the above considerations, in this manuscript, we develop a general framework for constructing PIRNN models that can mitigate poor control performance engendered by plant-model mismatch and system disturbances. Specifically, an error-triggered PIRNN modeling framework is developed for a general class of nonlinear processes subject to

parameter uncertainty. While the forward modeling approach is utilized to develop a PIRNN with a nominal mechanistic system model, the inverse technique is exploited to estimate the uncertain model parameters based on the latest process data acquired. The PIRNN model is then updated online based on the newly estimated process parameters using an error-triggered mechanism to better conform to the system dynamics subject to time-varying disturbance, and to provide a more robust and effective process control. A chemical reactor example with process disturbances will be used in the last section to demonstrate the superiority of the inverse PIRNN-enhanced online learning scheme developed in this work compared to the conventional purely data driven online update technique and the forward PIRNN model without online learning. Therefore, in the presence of model uncertainty, the inverse PIRNN modeling method should be implemented to efficiently update the physical-law-based regularization term to improve the training performance of PIRNN models.

■ PRELIMINARIES

Notation. The Euclidean norm of a vector is denoted by the operator $\|\cdot\|$. The transpose of a vector x is represented by x^T . Set subtraction is represented by the notation “\” where $A \setminus B := \{x \in \mathbb{R}^n \mid x \in A, x \notin B\}$. If the derivative of a function $f(\cdot)$, i.e., $f'(\cdot)$, exists and is continuous in its domain, then $f(\cdot)$ is said to belong to the differentiability class C^1 . The expected value of a random variable X is denoted by $\mathbb{E}[X]$, and the probability of an event A occurring is represented by $\mathbb{P}(A)$.

Class of Nonlinear Systems. In this work, a class of nonlinear dynamic systems described by the following first-order ordinary differential equations (ODE), is considered.

$$\dot{x} - F(x, u) := \dot{x} - f(x) - g(x)u = 0, \quad x(t_0) = x_0 \quad (1a)$$

$$y = x \quad (1b)$$

where $x \in \mathbb{R}^n$ represents the state vector of the nonlinear system. The solution of the first-order ODE is denoted by $x(t)$, the manipulated input vector is represented by $u \in \mathbb{R}^m$, and $y \in \mathbb{R}^n$ denotes the real-time sensor measurements of the state vector. Measurement noise is assumed to be negligible and will not be considered in this work. $u \in U := \{u_{\min} \leq u \leq u_{\max}\} \subset \mathbb{R}^m$ defines the constraint on the manipulated variable u , where $u_{\min}$ and $u_{\max}$ denote the lower and upper bounds (i.e., the physically achievable limits) of the manipulated input, respectively. $f(\cdot) \in \mathbb{R}^{n \times 1}$ and $g(\cdot) \in \mathbb{R}^{n \times m}$ are a sufficiently smooth vector and a matrix function, respectively. Without loss of generality, it is assumed that the initial time $t_0 = 0$, and $f(0) = 0$. The origin is therefore a steady-state of the nonlinear system of eq 1, with $(x_s^*, u_s^*) = (0, 0)$ representing the steady-state system state, and manipulated input vectors, respectively. Furthermore, it is also assumed that real-time sensor measurements of the complete system states $x \in \mathbb{R}^n$ are readily available. In order to guarantee stabilizability of the nonlinear system of eq 1 with full-state measurements, it is assumed that a stabilizing control law $u = \Phi(x) \in U$ (e.g., the universal Sontag control law²³) exists, and is capable of rendering the origin of the nonlinear system exponentially stable.

Physics-Informed (PI) RNN. In the perspective of traditional data-driven modeling approaches (e.g., machine learning modeling of nonlinear processes), accurate prediction performance is often contingent upon the presence of an

enormously large training data set. However, in real-world manufacturing systems, an exhaustively sampled process data may not always be readily available, especially for newly established production plants, and this can severely thwart the control performance of ML-based controller based on conventional purely data-driven modeling techniques. To alleviate these challenges, physics-informed (PI) learning introduces an alternative modeling framework that seamlessly incorporates mathematical physics models into the neural network's learning.⁷ In its essence, the PI learning methodology facilitates knowledge and data assimilation by extracting key information from both the observational data and the physical laws (e.g., process dynamics governed by eq 1) by incorporating the governing ODEs as additional penalty terms into the loss function of a machine learning model. In regression problems, an RNN is typically trained by modifying its weight parameters to minimize a stipulated cost function that computes the mean squared error (MSE) between the actual data labels and the model predictions. To ensure a more directed and effective training process, additional regularization constraints are often introduced to the cost function to enhance learning performance. A widely adopted regularization technique is L2 regularization (also termed ridge regression or weight decay), which serves to represses the magnitude of weight parameters by penalizing the L2 norm of network weights, thus mitigating the poor generalizability ascribed to overfitting.²⁴ Following our previous work²² that developed physics-informed RNN models for the nonlinear system of eq 1, we propose an inverse physics-informed ML modeling method in this section to estimate unknown parameters in the nonlinear system of eq 1.

This section outlines the implementation of PIRNN models in solving the forward and inverse problems where process knowledge with respect to the underlying physical principles is assumed to be complete and incomplete, respectively.

Recurrent Neural Network (RNN) Model. In this work, RNN is used as a surrogate model for the nonlinear system of eq 1, and is mathematically represented as follows:

$$\dot{\tilde{x}} = F_m(\tilde{x}, u) := A\tilde{x} + \Psi^T z \quad (2)$$

where the RNN state and the manipulated input vectors are denoted by $\tilde{x} \in \mathbf{R}^{d_x}$, and $u \in \mathbf{R}^{d_u}$, respectively.

$$\begin{aligned} z &= [z_1, \dots, z_{d_x}, z_{d_x+1}, \dots, z_{d_x+d_u}] \\ &= [\sigma(\tilde{x}_1), \dots, \sigma(\tilde{x}_{d_x}), u_1, \dots, u_{d_u}] \in \mathbf{R}^{d_x+d_u} \end{aligned}$$

is a vector of both the RNN state $\tilde{x}$ and the manipulated input u . The nonlinear activation function (e.g., *ReLU* function $\sigma(x) = \max\{0, x\}$ and *tanh* function $\sigma(x) = (e^x - e^{-x})/(e^x + e^{-x})$) is represented as $\sigma(\cdot)$. The RNN weight parameters are expressed as coefficient matrices A and Ψ , which will be adjusted to minimize the specified loss function during training. It should be noted that throughout the manuscript, x denotes the states of the original nonlinear system of eq 1, and $\tilde{x}$ represents the states of the surrogate RNN model of eq 2. To further simplify the notation, θ is used to denote the matrix of all RNN weights to be optimized.

The development of an accurate RNN model usually demands an enormous amount of training data, which can be cumbersome and exorbitant to obtain in many practical ML tasks. Furthermore, although an RNN with an appropriate architecture is a universal surrogate that is capable of

approximating any nonlinear dynamic system to an arbitrary accuracy and on compact subsets of the state space for finite time, algorithmic learnability of the optimal network weight parameters is not guaranteed. In practice, searching for the optimal network weights and hyperparameters (e.g., number of layers and recurrent units) that best approximate a nonlinear system is a challenging and multifaceted problem, and is often dependent on various factors such as the availability of high-performance computing resources, and the selection of optimization algorithms. In this respect, enhancing the modeling performance of neural networks has been a long-standing research problem in the machine learning community, and considerable efforts have been devoted to devising novel network architecture and advanced optimization algorithms, and improving data acquisition and quality (e.g., via design of experiments). In the following sections, the PI technique, as proposed in ref 7 is adapted to improve the modeling performance of RNN for improved applicability in both forward and inverse problems.

Remark 1. To facilitate and simplify the discussion of approximating the nonlinear system of eq 1 with RNN, the mathematical representation of the RNN shown in eq 2 only considers an architecture comprising three layers: an input layer, a hidden layer, and an output layer. It should be noted that its actual development is not restricted to only one hidden layer, and can be extended to an arbitrary number of hidden layers for better modeling performance. Moreover, since the RNN model developed in this work serves to predict future system states given the current state measurements and manipulated inputs, its internal states would correspond directly to the process states output by the nonlinear dynamic system of eq 1 (when given the same initial states and manipulated inputs).

Regularization through Physics-Guided Loss Function. In the PI deep learning modeling paradigm, RNN is trained by introducing appropriate physics-based terms to the loss function as soft constraints, and this explicitly guides the learning algorithm to converge toward predictions that comply with the underlying mechanistic principles in forward problems or to infer unknown parameters from a limited set of observed data in inverse problems. Intuitively, the incorporation of a priori physics-based loss terms subtly harnesses the physical knowledge captured in mathematical mechanistic models and observational data for solving forward and inverse problems, respectively.

Forward Problem. Before we present the formulation of inverse PIRNN models to estimate unknown process parameters for the nonlinear system of eq 1 subject to model uncertainties, we first provide a brief recap of the forward PIRNN modeling method developed in our previous work.²² In forward problems, the augmented loss function (i.e., with the inclusion of physics-based terms) reinforces the neural network in yielding predictions that are consistent with the mechanistic model (i.e., the governing equation of eq 1 of the dynamic nonlinear system). In this regard, it is desirable for the RNN output to conform to the process dynamics outlined by the underlying physical laws, as expressed by the ODE of eq 1, to ensure satisfactory approximation performance. We consider the extreme case where the forward problem is solved without any data assimilation (i.e., assuming that no observational data are present). To express the notion of the PI learning paradigm mathematically for solving forward problems, a new function $\mathcal{G}$ is first defined to be the equivalent of eq 1a.

$$\mathcal{G}(\tilde{x}, u) := \tilde{x} - F(\tilde{x}, u) \quad (3)$$

where the output of the RNN is denoted by $\tilde{x}$. The inputs to the RNN consist of the initial system state vector x_0 , and the manipulated input u . Furthermore, $\tilde{x} = h(z; \theta)$, where $z = [x_0, u]$ represents the RNN input vector, and $h(z; \theta)$ therefore denotes the approximation of the solution x of the nonlinear system of eq 1a for time $0 \leq t \leq \Delta$ with a given x_0 and u , and the RNN weights θ . Consequently, as the PI learning paradigm facilitates the training of RNN surrogates to output predictions that approximate the solution x of the governing ODE of eq 1a (i.e., $\tilde{x} = h(z; \theta) \approx x$), the following expression should therefore be valid in the absence of model uncertainties.

$$\mathcal{G}(\tilde{x}, u) := \tilde{x} - F(\tilde{x}, u) \approx 0, t \in [0, \Delta] \quad (4)$$

The PI-based RNN (PIRNN) $h(z; \theta)$ is trained using a finite set of collocation points, and an optimization algorithm such as Adam. The collocation points comprise only the input data to the RNN (i.e., the initial state vector x_0 and manipulated input u) that are uniformly sampled from the designated operating regions where the ODE approximation (eq 4) holds. In this work, the PIRNN is constructed to predict the state trajectory $\tilde{x}(t)$ and the corresponding $\tilde{x} - F(\tilde{x}, u, \lambda)$ for one sampling period (i.e., Δ) forward. The PI-based loss function of the RNN is represented by eq 5.

$$\begin{aligned} \text{MSE} = & \gamma \frac{1}{N_G} \sum_{n=1}^{N_G} |x_n(t_{G,0}, u_{G,n}) - \tilde{x}_n(t_{G,0}, u_{G,n})|^2 \\ & + \frac{1}{N_G} \sum_{n=1}^{N_G} \frac{1}{N_T} \sum_{i=0}^{N_T} |\mathcal{G}(\tilde{x}_n(t_{G,i}, u_{G,n}), u_{G,n})|^2 \end{aligned} \quad (5)$$

where the number of collocation points and outputs of RNN at time $0 \leq t \leq \Delta$ are denoted by N_G , and N_T , respectively. γ denotes a constant coefficient that seeks to balance the scale of the two loss terms in MSE of eq 5. The value of γ is typically defined by the user to adjust the interplay between the two contributing MSE loss terms and to facilitate the training of RNN. With a slight abuse of notation, $x_n(t_{G,i}, u_{G,n})$ is used to denote the solution of the ODE of eq 1a for time $0 \leq t \leq \Delta$ under the initial system state $x_n(t_{G,0})$ at $t = t_0$, and the manipulated input $u = u_{G,n}$ which remains constant for all $t \in [0, \Delta]$. Since the PIRNN model will be incorporated in the MPC design, for which the control actions are implemented in a sample-and-hold fashion (i.e., $u(t) = u(t_k), \forall t \in [t_k, t_k + \Delta]$), the manipulated input u remains constant for each sampling period in the “physics-related” loss function to be consistent with the implementation of MPC. The output of the RNN at time $t_{G,i}$, and under the manipulated input $u_{G,n}$ is denoted by $\tilde{x}_n(t_{G,i}, u_{G,n})$.

Remark 2 As discussed in ref 22 the performance of forward PIRNN can be further improved by introducing data points that have been directly acquired from the actual nonlinear system into network training by augmenting the loss function of eq 5 with a standard loss function that minimizes the error between predicted output and labeled output. Specifically, the augmented MSE loss function can be represented as $\text{MSE} = w_X \text{MSE}_X + \text{MSE}$, where the additional term $\text{MSE}_X := \frac{1}{N_X} \sum_{n=1}^{N_X} \frac{1}{N_T} \sum_{i=0}^{N_T} |x_n(t_{X,i}, u_{X,n}) - \tilde{x}_n(t_{X,i}, u_{X,n})|^2$ represents the standard MSE loss with respect to the collected process data, and w_X is the weight coefficient that balances the

scale of the two MSE loss terms. The weight coefficient will be tuned by trial-and-error to achieve the desired training performance. Note that the collocation points in eq 5 should be sampled predominantly from operating regions that are beyond the range of real plant data. The fusion of data- and physics-driven RNN modeling thus maximizes the model performance which adheres to the dynamic behavior governed by the ODE while simultaneously engendering the best possible fit to plain training data points collected.

Inverse Problem. While the forward problem of PIRNN can improve the generalization performance of RNN models by incorporating the mechanistic model of eq 1a into the loss function and the collocation points which are inputs to the RNN that consist of only the initial state vector x_0 and manipulated input u , it is noted that the performance of PIRNN models is highly dependent on the accuracy of the mechanistic model. Therefore, in the presence of model uncertainties such as disturbances and time-varying system dynamics, the PIRNN model developed using the forward problem of eq 5 may not be able to capture the underlying process dynamics. To address this issue, we develop the inverse problem of PI-based loss function, seeking to reconcile and uncover information embedded in a limited set of observed data and the incomplete physics models with unknown parameters. This allows data-driven discovery of meaningful solutions to the unknown parameters in the governing ODE. It is assumed that (1) the model structure of the original nonlinear first-principles model used in the physical-law-based regularization term of eq 5 remains unchanged, and (2) we know in advance which process variables are subject to uncertainty due to lack of direct measures. Under the above assumptions, observational data can be used to capture the variation of uncertain parameters. Unlike robust controller design for processes that account for uncertainty (e.g., refs 25–28), the inverse PIRNN modeling method in this work is to estimate the uncertain parameters based on real-time process data. To put it formally, a similar function $\mathcal{G}$, which is comparable to that of eq 4, is defined as follows.

$$\mathcal{G}(\tilde{x}, u, \lambda) := \tilde{x} - F(\tilde{x}, u, \lambda) \approx 0, t \in [0, \Delta] \quad (6)$$

where the notations follow those of eq 4. Note that λ denotes the unknown parameters in the incomplete physics model, rather than an added term that represents the process disturbance to be estimated, and it can be informatively estimated from the observational data using PIRNN. Additionally, the model uncertainty considered in this work is time-varying and is due to the uncertainty in model parameters.

Contrary to the forward problem, observational data are essential for solving the inverse problem. Specifically, the MSE function takes the following form.

$$\begin{aligned} \text{MSE} = & \eta \left[\frac{1}{N_X} \sum_{n=1}^{N_X} \frac{1}{N_T} \sum_{i=0}^{N_T} |x_n(t_{X,i}, u_{X,n}) - \tilde{x}_n(t_{X,i}, u_{X,n})|^2 \right] \\ & + \frac{1}{N_G} \sum_{n=1}^{N_G} \frac{1}{N_T} \sum_{i=0}^{N_T} |\mathcal{G}(\tilde{x}_n(t_{G,i}, u_{G,n}), u_{G,n}, \lambda_{G,n})|^2 \end{aligned} \quad (7)$$

where the notations follow those of eq 5. The subscripts X and G represent the loss terms with respect to conventional MSE between the observed data and the model predictions in a typical regression problem, and the physics-based loss which is denoted by eq 6, respectively. N_X represents the number of

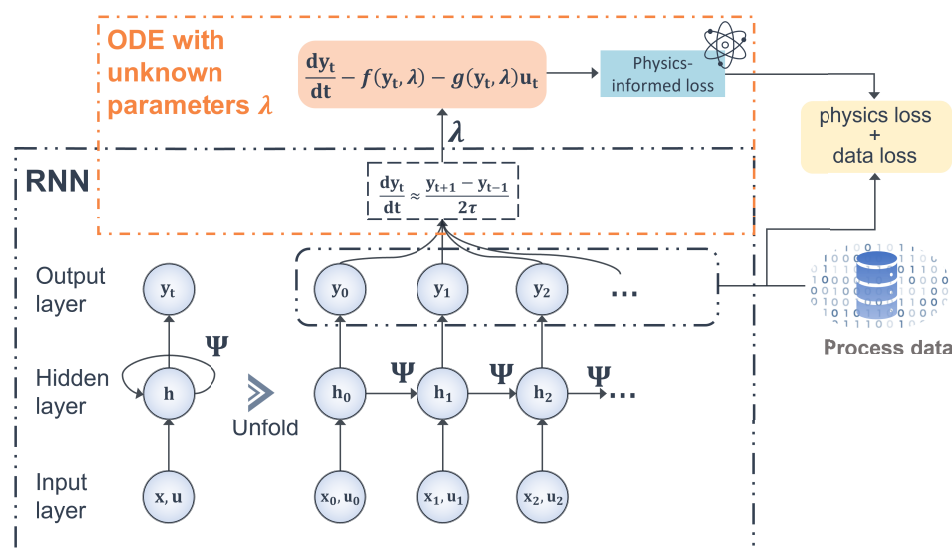


Figure 1. Schematic of the PIRNN model with physics-guided regularization terms embedded into the loss function for solving inverse problems. The outputs of the PIRNN model need to match the empirical process data captured to achieve the minimum data loss. The derivatives are then computed by the finite difference method using the outputs of the PIRNN model, where a time step τ that could be the integration time step of h_c is employed in their calculations. The neural network optimizes the unknown parameters λ to attain the minimum PI loss.

training data (i.e., the number of dynamic state trajectories captured from sensors). η is a constant coefficient that regulates the interaction between the two contributing MSE loss terms and facilitates the discovery of the unknown parameter λ . The first loss term in eq 7 corresponds to minimizing the MSE loss between the predicted states and the actual process data collected, and the second loss term attempts to estimate the uncertain process parameter λ_G that best conforms to the partial mathematical structure of the dynamic system. In general, more emphasis is placed on minimizing the data-driven MSE loss (i.e., the first loss term in eq 7) to ensure a good fit to the observed data, and to facilitate the estimation of the uncertain process parameter λ_G (i.e., the second loss term in eq 7) that best describes the inferred process dynamics. In practice, the choice of the weight coefficient η in eq 7 depends on the number and the quality of training samples. For example, if the training data are noisy, less emphasis (i.e., a small value of η) should be placed on minimizing the data-driven MSE loss in eq 7 to reduce overfitting on noisy data and improve the generalization performance by enforcing the physical law.

In its application to inverse problems, the PI learning paradigm resembles that of multitask learning where the RNN model is contemporaneously constrained to achieve the best possible fit to the observed data (e.g., plain training data from sensor measurements), and to discover the most plausible values of the unknown parameter λ that conform to the physics-based constraints of eq 6. Therefore, the PI approach integrates observational data with partial a priori theoretical knowledge in solving inverse problems by optimizing the network weights θ and the unknown physics model parameter λ by minimizing the combined MSE loss function of eq 7. Figure 1 illustrates the general structure of a PIRNN for solving inverse problems that involve nonlinear dynamic systems. Specifically, in addition to computing the data loss, the predicted states are also used to estimate the time derivatives involved in the mechanistic model by the finite difference method, and this forms the basis for minimizing

physics-based loss, and for estimating the unknown process parameter λ . Along with the predicted states $\tilde{x}$, the PIRNN model also outputs the corresponding $\tilde{x} - F(\tilde{x}, u, \lambda)$, which is involved in the subsequent calculation of the physics loss term in eq 7. λ is incorporated as a network weight in the output layer, and will be optimized during model training to minimize the physics-based loss. $\tilde{x}$ is approximated using the finite difference method on the predicted states $\tilde{x}$. The final λ determined is the solution to the inverse problem.

Remark 3. Since the focus of this study is to develop an inverse PIRNN modeling approach that can guide the training of PIRNN models using the updated first-principles model in a computationally efficient manner, the theory of observability for nonlinear systems with uncertain parameters is not investigated in this work. It is noted that a general way to study parameter observability is to treat the model parameter as a state variable whose time derivative is zero. There are various ways to check the observability, (e.g., refs 29–31) and perform estimation for nonlinear process uncertain parameters while accounting for observability, (e.g., refs 32,33). For example, an Abridged Gaussian Sum Extended Kalman Filter (AGS-EKF) for nonlinear state estimation under non-Gaussian process uncertainties was developed in ref 32 and a constrained AGS-EKF for constrained nonlinear systems with non-Gaussian measurement noises and process uncertainties was proposed in ref 33. Therefore, to ensure that the system with uncertain parameters is observable, one solution is to formulate observability criteria as a constraint of the optimization-based inverse PIRNN modeling process.

Remark 4. The inverse PIRNN modeling approach in this section is developed to address parameter uncertainty. However, if structural uncertainty is considered (e.g., the functional form of the process model is changed), then an additional disturbance term can be added to treat the variation as external disturbances, and perform approximation using real-time process data. The entire process model with the disturbance term will be used in the calculation of the physical-law-based regularization term in the inverse PIRNN

modeling. Therefore, in the presence of parameter uncertainty, we can directly approximate the uncertain parameters provided that the model structure remains unchanged, while in the presence of model structural uncertainty, an additional disturbance term can be added to account for the impact of uncertainty.

Implementation of PIRNN. In its implementation to solving both the forward and inverse problems, the PIRNN model is developed using PyTorch,³⁴ which is an open-source ML framework based on the Torch library, and is one of the most popular ML frameworks for researchers and practitioners. The inputs to the PIRNN model comprise the initial system state vector x_0 , and the manipulated input u . Specifically, the manipulated input u remains constant for each sampling period $t \in [t_0, t_0 + \Delta]$, where Δ represents the prediction horizon of the PIRNN model. In the forward problem, it is assumed that the empirical data are unavailable, and complete knowledge of the mathematical physics model is accessible. The PIRNN model is thus trained entirely on the collocation points only, which encompass solely the initial system state x_0 and the manipulated input u that are uniformly sampled across the entire stability region Ω_p and the physical limits of the manipulated inputs u in computing the corresponding MSE function of eq 5, and no labeled data are supplied to the PIRNN. It should be noted that numerical simulation of the governing ODE of eq 1 based on the collocation points is not needed in training the PIRNN model as the system dynamics are explicitly embedded in the second loss term of the MSE function of eq 5, where labeled output data are no longer required for learning the underlying physical principles.

On the contrary, in the inverse problem, it is assumed that a limited set of observational data is readily accessible and only a partial knowledge of the mathematical physics model is available (i.e., models containing unknown physical parameters λ). Empirical data are acquired during the system operation that consists of the initial system states x_0 , the manipulated inputs u , and the evolution of the system states x in one sampling period. The unknown parameter λ is then estimated based on the system dynamics embedded in the observed data and the formulation of the incomplete physical model by minimizing the corresponding MSE loss function of eq 7 via optimizing network weights θ and the unknown parameter λ .

Constant scaling factors γ and η are incorporated into the MSE loss functions of eq 5 and eq 7 for forward and inverse problems, respectively, to indicate the relative importance of the various loss terms, and to facilitate the training of PIRNN. Specifically, the computation of the respective first loss term in the MSE functions of eq 5 and eq 7 resembles that of the conventional MSE loss (i.e., the MSE between labeled output data and RNN predictions) in a typical ML regression task. However, the calculation of the second loss terms of the MSE functions of eq 5 and eq 7, respectively, stipulates the determination of the time derivative of $\tilde{x}$ (i.e., $\dot{\tilde{x}}$). As shown in Figure 1, the finite difference method can be exploited to estimate $\dot{\tilde{x}}$ by using the predicted outputs from the PIRNN directly. Depending on the nature of the problem of interest (i.e., a forward or an inverse problem), the time derivatives computed can be used to compute the corresponding physics-based loss in eq 5 and eq 7, respectively.

The implementation of a PIRNN for solving a forward problem is relatively straightforward for nonlinear systems, and has been discussed in detail in ref 22. The implementation of

PIRNN for solving an inverse problem (i.e., estimating uncertain process parameters in a nonlinear system) is presented in this section, as outlined by Algorithm 1. Specifically, the uncertain process parameter λ is first initialized as torch.tensor using 'torch.nn.Parameter(λ , requires_grad = True)', where 'requires_grad = True' indicates that the training algorithm is to keep track of the parameter gradient in forward propagation in order to optimize the value of λ in back-propagation. λ is subsequently added to the pretrained PIRNN model via the 'model.register_param()' method, and the model can now be updated with the latest process data acquired with respect to minimizing the loss function of eq 7.

It should be noted that, contrary to the PIRNN model constructed to solve a forward problem, which outputs only the predicted states $\tilde{x}$, an additional layer is added to the output layer of the PIRNN model to solve the inverse problem. The outputs of the PIRNN model in an inverse problem consist of the predicted states $\tilde{x}$ and the corresponding $\dot{\tilde{x}} = F(\tilde{x}, u, \lambda)$ to calculate the hybrid loss function of eq 7. The uncertain process parameter λ is incorporated as the network weight in the output layer of the PIRNN model, which will be optimized during training. { Note that the training process of inverse PIRNN models itself is an optimization problem that optimizes λ to minimize the loss function of eq 7. Therefore, if a sufficiently small loss value is achieved, it implies that the first-principles model with the derived λ approximates well the actual nonlinear system subject to parameter uncertainty. However, in the event that the loss value is not sufficiently small after enough training time due to, for example, insufficient real-time data, then the error-based update will trigger again at the next time step until the model with updated parameters can capture the latest dynamics well. Additionally, it should be clarified that data are not necessarily required for training a pretrained forward PIRNN model in Algorithm 1. In the worst-case scenario where no process data are available, we can still develop a forward PIRNN model purely based on collocation points generated by the first-principles model. In our previous work,²² we have demonstrated the applicability of the forward PIRNN modeling approach using first-principles knowledge only to the system without data.

Algorithm 1 Implementation of PIRNN for estimating uncertain process parameter λ of nonlinear dynamic systems using PyTorch

Require: a pre-trained PIRNN model by the forward approach and empirical system states captured by sensors

1. Initialize uncertain process parameters λ as torch.tensor using torch.nn.Parameter(λ , requires_grad=True)
2. Add the parameter λ to the PIRNN model by model.register_param(λ) method, which will be optimized during training
3. Update the pre-trained PIRNN with the latest empirical data acquired with respect to minimizing the loss function of Eq. 7

Remark 5. In addition to inverse PIRNN modeling approach, improving the existing first-principles model by updating its parameters could be one solution to improving model performance with limited data. The major difference between the proposed inverse PIRNN modeling method and the parameter estimation of first-principles models is that the proposed method develops a neural network model, which can represent a rich set of nonlinear functions; however, it is generally challenging to develop an accurate first-principles model for nonlinear processes without knowing its functional form.

Generalization Error of Inverse PIRNNs. Deriving an upper bound for the generalization error of a supervised

learning algorithm is one of the most important problems in machine learning modeling of nonlinear systems for model predictive controller, since the generalization error provides a measure of modeling accuracy on previously unseen data from the same distribution. In ref 22 we have developed a generalization error bound for the forward problem of PIRNN, where a new hypothesis $\tilde{h}(z; \theta)$ that satisfies $h(z; \theta) = (t - t_0)\tilde{h}(z; \theta) + x_0$, $z = [x_0 u]$ is employed to convert the loss function of eq 5 standard MSE loss function for training RNNs. In this section, we provide a generalization error analysis for the inverse problem of PIRNN that is developed to estimate unknown parameters in the nonlinear system of eq 1 subject to model parameter uncertainties. Specifically, the generalization error for the inverse PIRNNs with the loss function of eq 7 is defined as follows:

$$R_D(\theta) := \eta \mathbb{E}_{z \sim Z} [L(\tilde{x}_n, x_n)] + \mathbb{E}_{z \sim Z} [L(\mathcal{G}(h(z; \theta), u, \lambda), 0)] \quad (8)$$

where the first term represents the expected loss between the predicted state $\tilde{x}_n$ and the observed state x_n (i.e., labeled data), and the second term represents the expected loss associated with the estimation of unknown parameters of eq 6. Since the loss function used in eq 8 contains two loss terms, which is not in the general form of the loss functions (e.g., a single MSE function) used in regression problems, the generalization error results derived in refs 35 and 36 for standard RNN models cannot be directly applied here. However, as discussed in the section “Inverse Problem”, by constructing the PIRNN for the inverse problem with an output layer consisting of both the predicted states $\tilde{x}$ and the corresponding physics loss term $\tilde{x} - F(\tilde{x}, u, \lambda)$, the loss function of eq 6 can be written as a single MSE loss term penalizing both $\tilde{x}$ and $\tilde{x} - F(\tilde{x}, u, \lambda)$, where λ is taken as one of the PIRNN weights to be optimized during the training process. Additionally, the weight coefficient η in eq 8 can also be considered as a scaling factor in the PIRNN weights, and thus, the generalization error for the new loss function is represented as follows:

$$R_D(\theta) = \mathbb{E}_{z \sim Z} [L(\tilde{w}, \bar{w})] \quad (9)$$

where $\tilde{w} = [\tilde{x}_n \quad \mathcal{G}(h(z; \theta), u, \lambda)]$ represents the augmented output vector predicted by the inverse PIRNN model, and $\bar{w} = [x_n \quad 0]$ represents the true labeled output. In this way, training the inverse problem of PIRNNs follows the same training process for any standard RNNs, and therefore, the following generalization error results can be readily obtained for the inverse PIRNNs.

Proposition 1 (c.f., Theorem 1 in ref 35) Consider the PIRNN model of eq 2 that can be written in the discrete-time form $\mathbf{h}_t = \sigma_h(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$, $\mathbf{y}_t = \sigma_y(\mathbf{V}\mathbf{h}_t)$, where $\mathbf{x}_t$, $\mathbf{h}_t$, and $\mathbf{y}_t$ represent the PIRNN inputs, hidden states, and outputs at the t_{th} time step, $t = 1, \dots, T$, respectively. Given a data set $S = (z_{i,t}, \bar{w}_{i,t})_{t=1}^T$, $i = 1, \dots, m$ with m i.i.d. data samples, $i = 1, \dots, m$, we use $\mathcal{L}_t$ to denote the loss function class in eq 9 associated with the class of PIRNN functions $h \in \mathcal{H}$ that are developed for the inverse problem of eq 6 using the training samples S . If the PIRNN model is developed with bounded inputs and weight matrices, i.e., $\|\mathbf{x}_{i,t}\| \leq B_X$, for all $i = 1, \dots, m$ and $t = 1, \dots, T$, and $\|\mathbf{W}\|_F \leq B_{W,F}$, $\|\mathbf{V}\|_F \leq B_{V,F}$, $\|\mathbf{U}\|_F \leq B_{U,F}$, where $\|\cdot\|_F$ de-

notes the Frobenius norm, then with probability at least $1 - \delta$ over S , the following inequality holds for the PIRNN models:

$$\mathbb{E}[L(\tilde{w}, \bar{w})] \leq \frac{1}{m} \sum_{i=1}^m L(\tilde{w}_i, \bar{w}_i) + O\left(L_r d_y \frac{MB_X(1 + \sqrt{2 \log(2)t})}{\sqrt{m}}\right) + 3\sqrt{\frac{\log\left(\frac{2}{\delta}\right)}{2m}} \quad (10)$$

where $M = \frac{1 - (B_{U,F})^t}{1 - B_{U,F}} B_{W,F} B_{V,F}$ is the product of the PIRNN weight matrix bounds, L_r is the local Lipschitz constant for the loss function of eq 9, and d_y is the PIRNN output dimension.

Note that unlike the standard RNN models developed for the nonlinear system of eq 1 which include predicted states only in the output layer,³⁵ the PIRNNs for estimating unknown parameters in the system of eq 1 are developed with an augmented output vector $\tilde{w}$ that includes both predicted states and predicted time derivative of states, and thus, the output dimension d_y for PIRNNs is two times the dimension of states $x \in \mathbb{R}^n$, i.e., $d_y = 2n$. Additionally, the PIRNN model is developed following the standard RNN structure of eq 2, where the continuous-time system of eq 2 can be readily converted to its discrete-time form shown in the statement of Proposition 1 by unfolding the PIRNN structure over time. It has been discussed in ref 1 that the unfolded, discrete-time representation of RNN models is consistent with the general training process using sampled data collected from experiments or simulation, while the continuous-time PIRNN model is adopted to simplify the formulation of PIRNN-based MPC and closed-loop stability analysis for the nonlinear system of eq 1 represented in a continuous-time form. However, it should be pointed out that the continuous- or discrete-time representation of RNN models does not affect the closed-loop stability results, provided that the same numerical methods are utilized to compute the modeling error, characterize the closed-loop stability region, and calculate the constraints of PIRNN-based MPC.

■ ERROR-TRIGGERED PIRNN ONLINE UPDATE

Industrial production systems are generally vulnerable to unknown disturbances (e.g., due to changes in impurity concentrations of raw materials and catalyst deactivation). Consequently and inevitably, the presence of significant model-plant mismatch greatly deteriorates the control performance of model-based controllers such as MPC, thereby engendering suboptimal attainment of the desired system set points. PIRNN predictive models developed based on nominal mechanistic process models are often susceptible to considerable deviations from the actual system dynamics, and on-the-fly model update is therefore imperative to obtaining a desired model fidelity and a more effective control.

In general, conventional ML paradigms solely hinge upon offline learning where the predictive model is trained on a sufficient amount of data acquired from past operations before deployment, and postdeployment model update is typically not executed.³⁷ Online learning is a fundamentally different paradigm that is based on on-the-fly and incremental training of predictive models using an inflow of new operational data in a sequential manner. The online learning framework thus inherently resolves the aforementioned major issues pertaining to the offline learning algorithms by exploiting continuous

model improvement based on the most recent process data captured. Specifically, online learning adopts a finite set of active training data, and the online predictive model update algorithm is only triggered when a significant model-plant mismatch is present (e.g., MSE of model predictions and actual observed dynamics exceeds a specific threshold for an extended number of sampling periods). In addition, the predictive model may only be able to sufficiently capture the new process dynamics through several rounds of online update, and is thus incapable of achieving a satisfactory control in a timely manner. A possible solution is to enrich and augment the training data at the first time instance when the online learning is triggered by first estimating the new process parameters with the latest operational data obtained in an inverse manner, followed by updating the network weights θ with the new process parameters found in a forward manner. In this section, we introduce an enhanced PIRNN-based error-triggered online learning strategy that seeks to maximize the effectiveness of online update.

PIRNN-Enhanced Online Learning. An MPC scheme using PIRNN model is considered where real-time measurements of all system states x are readily accessible. To improve the closed-loop performance under MPC in the presence of unknown disturbances, a PIRNN-based error-triggered online learning mechanism is incorporated into the PIRNN-MPC scheme to maximize its prediction accuracy by estimating the numerical values of the altered process parameters in an inverse manner, and updating the model using the latest parameters found in a forward manner. Specifically, the triggering mechanism for online model update is based on computing the MSE between the predicted states and the measured states, which is defined by the following moving-horizon error metric $E_{RNN}(t_k)$.

$$E_{RNN}(t_k) = \frac{\|\tilde{x}'(t_k) - x'(t_k)\|^2}{n} \quad (11)$$

where the normalized system states for the sampling period t_k , subject to an identical sequence of control actions, are denoted by $\tilde{x}'(t_k)$ and $x'(t_k)$ for those predicted by the PIRNN model, and those acquired from the actual nonlinear dynamic system governed by eq 1, respectively. n represents the number of system states involved in calculating the MSE $E_{RNN}(t_k)$. Specifically, system states x_0 are measured and supplied to the PIRNN model at the beginning of each sampling period, and E_{RNN} is computed (for one sampling period) based on MSE between the predicted states (by the PIRNN model) and sensor measurements (obtained at the end of the sampling period).

In practice, it is often more prudent to garner process data from at least a few sampling times before invoking an online update of the PIRNN model to avoid overfitting due to data scarcity. The update of the PIRNN model is therefore triggered only if the cumulative error $E_{RNN}(t_k)$ exceeds a predefined threshold E_T (eq 12) consecutively for a stipulated number of sampling times.

$$E_{RNN}(t_k) \geq E_T \quad (12)$$

where E_T is the error threshold that can be determined via extensive closed-loop simulations such that the update is triggered at appropriate times to achieve a balance between update frequency and data-storage burden. The PIRNN-enhanced online learning mechanism is outlined in Algorithm

2, where the PIRNN model update is only triggered if eq 12 is satisfied for at least a few sampling times, and Algorithm 1 is embedded in Algorithm 2 as one of the steps to estimate the uncertain parameters.

Algorithm 2 PIRNN-enhanced error-triggered online learning mechanism

Require: Import pre-trained PIRNN model into MPC

```

while System is running do
  if  $E_{RNN} \geq E_T$  (Eq. 12) successively for a stipulated number of sampling times then
    (1.) Activate PIRNN-enhanced online update mechanism, where the model uncertainty is first quantified following the steps outlined in Algorithm 1
    (2.) Update the PIRNN model using the newly estimated model parameters
  else
    PIRNN model update is not triggered
  end if
end while

```

Remark 6. If the estimated uncertain parameter λ is not sufficiently accurate using the error-triggered online learning mechanism due to insufficient training data or training time, the cumulative error will exceed the predefined threshold after one update, which triggers the error-based update again at the next sampling step until the model with updated parameters can capture the latest dynamics well. In the event that the estimation of the uncertain parameter λ is still not accurate after several rounds of online update, the control performance may deteriorate using the inverse PIRNN model. One way to address this issue is to design a robust controller that accounts for the variation of process disturbances (e.g., lower and upper bounds), and use this predesigned robust controller as a backup controller to stabilize the system when the inverse PIRNN model with several rounds of online update cannot capture the latest dynamics well. Additionally, the value of the error threshold E_T should be carefully chosen such that the RNN model is only updated when “significant deviations” between the predicted values and the actual measurements are present. An underlying assumption for the error-triggered mechanism to be applied effectively in practice is that the time-varying process disturbance/parameter uncertainty does not vary frequently such that it allows some time for PIRNN to learn with sufficient data. This assumption is fair, as in general the parameter uncertainty or process disturbances do not change abruptly at every time step, which is different from the sensor noise that may change randomly and frequently.

PIRNN-BASED PREDICTIVE CONTROL

In this section, we incorporate the PIRNN model outlined in the previous section into the design of model predictive controllers (MPC) to optimize process performance while maintaining closed-loop stability. Specifically, for any initial condition $x_0 \in \Omega_p$, a Lyapunov-based model predictive controller (LMPC) is developed to drive the closed-loop state of the nonlinear system of eq 1 to the steady-state set point while maintaining the state in the stability region Ω_p for all times.

The Lyapunov-based model predictive control (LMPC) using the PIRNN model of eq 2 is utilized to stabilize the nonlinear system of eq 1 in the stability region. The formulation of the PIRNN-based LMPC optimization problem is given as follows:¹

$$\mathcal{J} = \min_{u \in S(\Delta)} \int_{t_k}^{t_{k+N}} (\tilde{x}^T Q \tilde{x} + u^T R u) dt \quad (13a)$$

$$\text{s. t. } \quad \tilde{x}(t) = F_{nn}(\tilde{x}(t), u(t)) \quad (13b)$$

$$u(t) \in U, \forall t \in [t_k, t_{k+N}] \quad (13c)$$

$$\tilde{x}(t_k) = x(t_k) \quad (13d)$$

$$\begin{aligned} \dot{V}(x(t_k), u) &\leq \dot{V}(x(t_k), \Phi_{nn}(x(t_k))), \\ \text{if } x(t_k) &\in \Omega_\rho \setminus \Omega_{\rho_{nn}} \end{aligned} \quad (13e)$$

$$V(\tilde{x}(t)) \leq \rho_{nn}, \forall t \in [t_k, t_{k+N}), \text{ if } x(t_k) \in \Omega_{\rho_{nn}} \quad (13f)$$

where $\tilde{x}$ is the predicted state trajectory, $S(\Delta)$ is the set of piecewise constant functions with period Δ , N is the number of sampling periods in the prediction horizon, and $\dot{V}(x, u)$ represents $\frac{\partial V(x)}{\partial x}(F_{nn}(x, u))$. Ω_ρ denotes the closed-loop stability region for the nonlinear system of eq 1, which is typically defined as a level set of the Lyapunov function in Lyapunov-based MPC. In the optimization problem of eq 13, the objective function of eq 13a is the integral of the cost function $l(\tilde{x}, t) = (\tilde{x}^T Q \tilde{x} + u^T R u)$ over the prediction horizon, where $l(0, 0) = 0$ and $l(\tilde{x}, t) > 0, \forall (\tilde{x}, t) \neq (0, 0)$. The constraint of eq 13b is the PIRNN model of eq 2 that is used to predict the states of the closed-loop system. Equation 13c defines the input constraints applied over the entire prediction horizon. Equation 13d defines the initial condition $\tilde{x}(t_k)$ of eq 13b, which is the state measurement at $t = t_k$. The constraint of eq 13e forces the closed-loop state to move toward the origin if $x(t_k) \in \Omega_\rho \setminus \Omega_{\rho_{nn}}$, where $\Omega_{\rho_{nn}}$ is a small neighborhood around the origin. However, if $x(t_k)$ enters $\Omega_{\rho_{nn}}$, the states predicted by the PIRNN model of eq 13b will be maintained in $\Omega_{\rho_{nn}}$ for the entire prediction horizon. The LMPC of eq 13 is implemented in a sample-and-hold fashion, that is, an optimal input trajectory $u^*(t), t \in [t_k, t_{k+N})$ is obtained by solving the LMPC optimization problem of eq 13 at each sampling time, from which only the control action for the first sampling period of the prediction horizon will be applied. The objective of the LMPC of eq 13 is to achieve closed-loop stability for the nonlinear system of eq 1 in the sense that the closed-loop state is bounded in the stability region Ω_ρ for all times, and can ultimately be bounded in a terminal set $\Omega_{\rho_{min}}$ that is a superset of $\Omega_{\rho_{nn}}$ designed accounting for the impact of modeling error and sample-and-hold implementation.

Recursive feasibility and closed-loop stability of the nonlinear system of eq 1 under the LMPC of eq 13 can be proven by showing that (1) the stabilizing controller $u = \Phi_{nn}(x)$ is a feasible solution to the MPC of eq 13 at all times, (2) the closed-loop state can be driven toward the origin, and ultimately bounded in the terminal set under $u = \Phi_{nn}(x)$, and (3) due to the two Lyapunov-based constraints of eqs 13e–13f, the optimal solution solved by MPC will achieve an equally good closed-loop performance, if not better, than the stabilizing controller. In our previous work, we have proven closed-loop stability in the probability sense for the nonlinear system of eq 1 under the machine-learning-based MPC using a purely data-driven RNN model with a sufficiently small generalization error.³⁵ Since a similar generalization error bound is derived for the inverse PIRNN in the section “Generalization Error of Inverse PIRNNs” after converting the custom loss function of eq 8 standard single-MSE loss function of eq 9, the closed-loop stability analysis for the RNN closely follows the one in ref 35 and is omitted here. Interested readers may refer to section 4 in ref 35 for the detailed proof of closed-loop stability for the nominal system of eq 1, and section 3 in

ref 36 for the discussion of closed-loop stability for uncertain nonlinear systems with stochastic disturbances.

Application to a Chemical Process Example. In this section, a chemical process example is utilized to demonstrate the application of the PIRNN-LMPC scheme with the incorporation of the proposed enhanced PIRNN-based online learning mechanism. The PIRNN-LMPC scheme is deployed to drive and maintain the closed-loop system states x at their respective steady-state set points and within the stability region Ω_ρ at all times. Specifically, a continuous stirred tank reactor (CSTR) is employed in this work, and the description of the CSTR and its parameters can be found in ref 38 and is presented in the Appendix. The control objective of the PIRNN-LMPC scheme is to maintain the system states of the CSTR at an unstable steady-state $(C_A, T_k) = (1.95 \text{ kmol/m}^3, 402 \text{ K})$ by adjusting the manipulated inputs (i.e., C_{A0} and Q), where the steady-state values of the manipulated variables are set at $(C_{A0}, Q_s) = (4 \text{ kmol/m}^3, 0 \text{ kJ/h})$.

To showcase the efficacy of the enhanced PIRNN-based online learning mechanism in alleviating poor control performance ascribed to model–plant mismatch, in this work, model variations due to the following disturbances are considered: (1) variation of the feed flow rate F caused by upstream process disturbances, and (2) decreasing frequency factor k_0 of the reaction in CSTR due to catalyst deactivation.

Approximating CSTR Dynamic Model with PIRNN.

The development of the initial PIRNN model is entirely based on the governing physical laws expressed by the mass and energy balance equations of eqs 14a–14b. At this stage, it is assumed that the parameter values listed in Table 2 are representative and adequate in describing the true dynamics of the CSTR system. The approximation of the CSTR dynamic model (i.e., the governing equations of eq 14) with PIRNN is therefore framed as a forward problem outlined in the section “Forward Problem”. It should be noted that, as indicated by the first loss term in the MSE function of eq 5, the output of the PIRNN entails the system states $\tilde{x}_n(t_{G,0}, u_{G,n})$ at $t = t_0$, and this is found to enhance the convergence toward the solutions of the governing equation of eq 14. Moreover, it is further assumed that no labeled data are available (e.g., a new chemical process); therefore, the PIRNN model is trained solely based on the collocation points, for which the second loss term in the MSE function of eq 5 enforces the system dynamics governed by the a priori mechanistic model by restricting the network weights and biases to yield predictions that conform to the solutions of the governing ODE at the given initial conditions. The detailed training process for the PIRNN model is presented in the Appendix.

The resulting PIRNN attained a mean squared error (MSE) of approximately 6.8×10^{-4} on the test data set, indicating a satisfactory generalization performance for approximating the process dynamics governed by the ODEs of eq 14 within the stipulated operating ranges dictated by the stability region Ω_ρ for the system states $(\Delta C_A \text{ and } \Delta T)$, and the physical bounds for the manipulated inputs $(\Delta C_{A0} \text{ and } \Delta Q)$, respectively. Figure 2 shows the comparison of open-loop state profiles predicted by the PIRNN model with those obtained from a numerical solver using the explicit Runge–Kutta method of order 5(4) with an integration time step $h_c = 1 \times 10^{-3} \text{ hr}$. Note that the collocation points in the test set encompass only the initial conditions, i.e., the system states ΔC_A and ΔT at $t = t_0$, and the manipulated inputs ΔC_{A0} and ΔQ which are kept

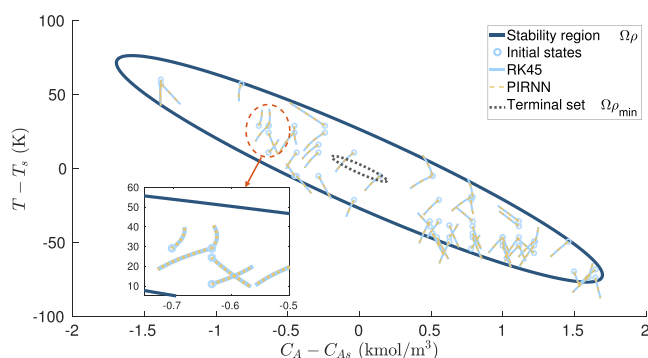


Figure 2. Comparison of open-loop state trajectories predicted by PIRNN (orange dashed line) with those obtained from the numerical solver using the explicit Runge–Kutta method of order 5(4) (light blue solid line), where the dynamic state trajectories from a total of 60 initial conditions consisting of the various pairs of system states (ΔC_A , ΔT) and manipulated inputs (ΔC_{A0} , ΔQ) that are randomly sampled from the test data set.

constant for one sampling period Δ , and are fed to the PIRNN model as piecewise constants with $\Delta C_{A0}(t) = \Delta C_{A0}(t_0)$ and $\Delta Q(t) = \Delta Q(t_0) \forall t \in [t_0, t_1]$, where $t_1 = t_0 + \Delta$. It is apparent that, as shown in Figure 2, the predicted states from the PIRNN model closely resemble those computed using the Runge–Kutta numerical algorithm, thereby demonstrating the superior generalization performance of the PIRNN model, and hence the PI learning approach in solving the forward problem.

Closed-Loop Simulation Results. In the closed-loop simulation, the dynamic CSTR system, which is described by the governing equations of eq 14, is employed to showcase the application of the PIRNN-based LMPC scheme of eq 13 equipped with the PIRNN-enhanced online learning algorithm in the previous section. The control objective of LMPC is to operate and maintain the CSTR system at an unstable equilibrium point $(C_A, T_s) = (1.95 \text{ kmol/m}^3, 402 \text{ K})$ by adjusting the manipulated inputs ΔC_{A0} and ΔQ based on the prediction of the PIRNN model. The effectiveness of the proposed PIRNN-enhanced online learning scheme is demonstrated through comparison with the control performances engendered by two other PIRNN-LMPC schemes with the traditional purely data-driven online update, and without any online update strategy, respectively. Therefore, the two additional PIRNN-LMPC schemes serve as benchmark cases to evaluate the performance of the PIRNN-based LMPC with the proposed online learning mechanism. Specifically, system disturbances caused by the feed flow rate F and the frequency factor k_0 are considered, where F gradually increases to $1.6F_{t=0}$ and $2.3F_{t=0}$ at $t = 0.09 \text{ hr}$ and 0.19 hr , respectively, with $F_{t=0}$ denoting the initial feed flow rate which adopts the value as stated in Table 2, and k_0 decreases to $0.8k_{0,t=0}$ and $0.3k_{0,t=0}$ at $t = 0.09 \text{ hr}$ and 0.19 hr , respectively, with $k_{0,t=0}$ representing the initial frequency factor that takes on the value as stated in Table 2. The initial PIRNN models incorporated into the various LMPC schemes in this section are identical, and are developed following the methodology illustrated in the section “Approximating CSTR Dynamic Model with PIRNN”. Moreover, note that in this work, it is assumed that measurement noise is negligible and is hence not considered for the data generated in the closed-loop simulations of the CSTR dynamic system. The error threshold E_T is specified as 1×10^{-4} . The nonlinear optimization problem of the LMPC schemes (i.e., eq

13) is solved using the PyIpopt optimizer with a sampling time of $\Delta = 1 \times 10^{-2} \text{ hr}$ and a total operation time of 0.3 hrs (i.e., 30 sampling periods).

The state-space trajectories and the state profiles of the CSTR dynamic system with an initial system state of $x_0 = (-1.5 \text{ kmol/m}^3, 70 \text{ K})$ under (1) PIRNN-LMPC without incorporating any online learning strategy (denoted by PIRNN_no_update), (2) PIRNN-LMPC with the conventional purely data-driven online update strategy (denoted by PIRNN_normal_update), and (3) PIRNN-LMPC equipped with the proposed PIRNN-enhanced online learning mechanism (denoted by PIRNN_enhanced_update), respectively, are shown in Figure 3, and Figure 4, respectively. As illustrated

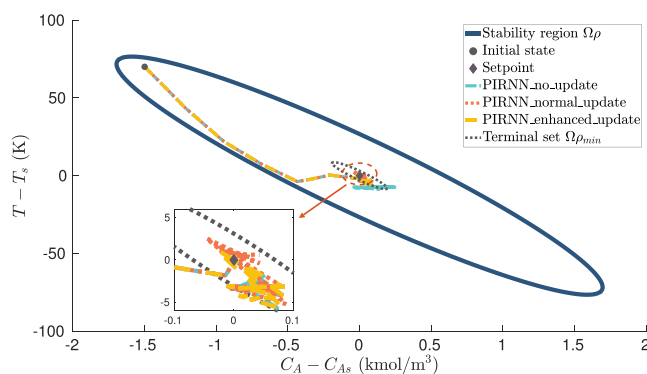


Figure 3. Comparison of closed-loop state trajectories under the PIRNN-LMPC without online learning (cyan dotted-dashed line), with conventional purely data-driven online update strategy (red dotted line), and with the proposed PIRNN-enhanced online learning scheme (orange dashed line), respectively, for the CSTR system starting from an initial condition of $(-1.5 \text{ kmol/m}^3, 70 \text{ K})$.

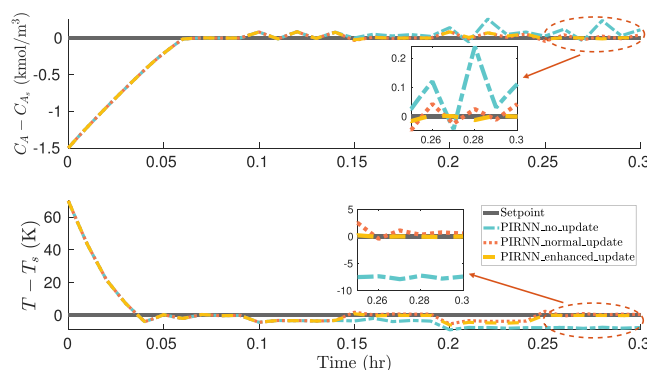


Figure 4. Comparison of the closed-loop state profiles under the PIRNN-LMPC without online learning (cyan dotted-dashed line), with conventional purely data-driven online update strategy (red dotted line), and with the proposed PIRNN-enhanced online learning scheme (orange dashed line), respectively, for the CSTR system starting from an initial condition of $(-1.5 \text{ kmol/m}^3, 70 \text{ K})$.

in both Figure 3 and Figure 4, the states of the CSTR system, prior to the onset of system disturbances, converge to a small neighborhood $\Omega_{\rho_{\min}}$ around the origin after approximately 6 sampling periods (i.e., $0.06 \text{ h} \leq t \leq 0.09 \text{ h}$) demonstrating the outstanding modeling and control performance of PIRNN-based LMPC schemes. However, the presence of system disturbances, which occur at $t = 0.09$ and 0.19 h , respectively, greatly hampers the control performance of PIRNN-LMPC, as manifested in the considerable deviations in the system states

from their steady-state set points after the first occurrence of system disturbance at $t = 0.09$ h in Figure 4. Furthermore, Figure 4 shows that the evolution of the closed-loop state profiles under LMPC without the online update of the PIRNN model (i.e., using the initial PIRNN model throughout the operation of CSTR) fails to converge to the steady-state set points (i.e., $(\Delta C_A, \Delta T) = (0, 0)$) post the introduction of system disturbances, while the two LMPC schemes incorporating the conventional data-based online update, and the proposed PIRNN-enhanced online update of the PIRNN model, respectively, successfully drive the closed-loop system states ΔC_A and ΔT toward and into a small neighborhood around the steady-state under the influence of system disturbances. Therefore, it is demonstrated that the incorporation of online learning is instrumental in mitigating the deterioration of control performance engendered by system disturbances and uncertainties.

Figure 5 shows the control actions adopted by the PIRNN-LMPC schemes without online learning (PIRNN_no_up-

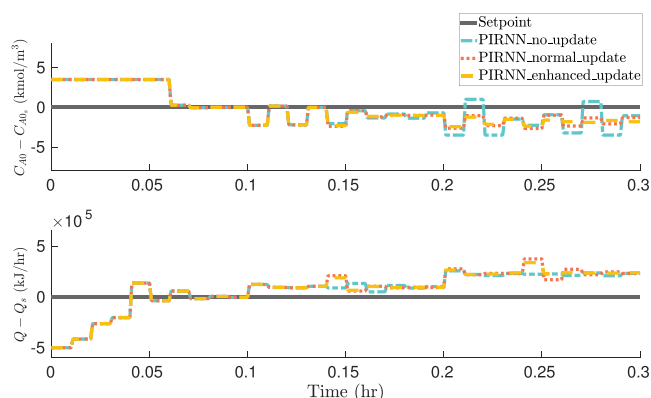


Figure 5. Comparison of the closed-loop manipulated inputs under the PIRNN-LMPC without online learning (cyan dotted-dashed line), with conventional purely data-driven online update strategy (red dotted line), and with the proposed PIRNN-enhanced online learning scheme (orange dashed line), respectively, for the CSTR system starting from an initial condition of $(-1.5 \text{ kmol/m}^3, 70 \text{ K})$.

date), with the conventional data-driven online update strategy, and with the proposed PIRNN-enhanced online learning mechanism, respectively. Figure 6 shows the evolution of the moving-horizon error $E_{RNN}(t)$ for the closed-loop CSTR system under the three PIRNN-LMPC schemes mentioned above, respectively. Specifically, prior to the occurrence of the first disturbance at $t = 0.09$ hr, the trajectory of system state profiles, control outputs, and the MSE error $E_{RNN}(t)$ are identical, as shown in Figure 4, Figure 5, and Figure 6, respectively. Furthermore, due to the remarkable approximation performance of the PIRNN model, the MSE errors $E_{RNN}(t)$ for $t \leq 0.09$ hr are significantly lower than the stipulated error threshold E_T of 1×10^{-4} , as illustrated in Figure 6. Therefore, both the system states ΔC_A , ΔT and the manipulated inputs ΔC_{A0} and ΔQ converge to and remain within a small neighborhood around their steady-state values (i.e., $x = (0, 0)$) smoothly and quickly after only 6 sampling periods for $0.06 \text{ hr} \leq t \leq 0.09 \text{ hr}$, as shown in Figure 4 and Figure 5, respectively.

However, at the first time instance when the system disturbances transpire at $t = 0.09$ hr, noticeable deviations of the system states from their set points occur, as shown in

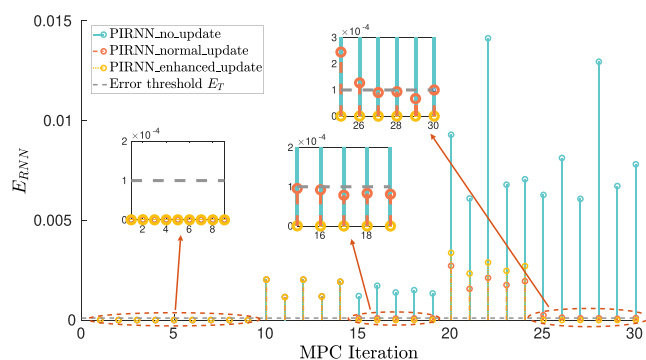


Figure 6. Comparison of the moving horizon MSE error of E_{RNN} at every sampling period during the CSTR operation under the PIRNN-LMPC without online learning (cyan dotted-dashed line), with conventional purely data-driven online update strategy (red dotted line), and with the proposed PIRNN-enhanced online learning scheme (orange dashed line), respectively, for the CSTR system starting from an initial condition of $(-1.5 \text{ kmol/m}^3, 70 \text{ K})$, where the error thresholds E_T is specified at 1×10^{-4} .

Figure 4. As further exhibited by the high MSE errors $E_{RNN}(t)$ that exceed the error threshold E_T immediately after the onset of system disturbances at $t = 0.09$ hr (Figure 6), the PIRNN model is deemed inadequate in describing the new system dynamics, and therefore, the PIRNN-LMPC controllers are incapable of maintaining the system states at their desired set points ensuing $t = 0.09$ hr (Figure 4). Ascribing to the presence of significant model-plant mismatch post $t = 0.09$ hr, the MSE errors $E_{RNN}(t)$ exceed the threshold $E_{RNN}(t)$ for 5 sampling periods, and this triggers the first online update at $t = 0.14$ hr. Consequently, as shown in Figure 4, the fine-tuned PIRNN model under the traditional data-based online update strategy is capable of making accurate predictions of the CSTR system states under the influence of system disturbances, and ultimately driving and maintaining the system states at their desired set point post the first online update (i.e., $t \geq 0.15$ hr).

For the online update using the proposed PIRNN-enhanced online learning mechanism, the process data acquired from the preceding 5 sampling periods are used first to estimate the system disturbance caused by the parametric drifts in the feed flow rate F and the frequency factor k_0 , respectively. The uncertain CSTR system after the first introduction of system disturbances (i.e., at $t = 0.09$ hr) is reparameterized by using the approach outlined in the section “Inverse Problem” for solving the inverse problem. Specifically, epoch = 600, optimizer = *Adam*, and learning rate = 1×10^{-3} are exploited in estimating the parametric drifts induced by the system disturbance in F , and k_0 , respectively, at $t = 0.09$ hr. To account for and optimize additional network parameters λ , which are utilized to quantify system disturbances, a relatively larger number of epochs is used (i.e., 600 epochs) as compared to that of the traditional data-driven online learning approach (i.e., 300 epochs). The estimation of the parametric drifts is framed as a multitask learning problem, where the PIRNN model is contemporaneously trained to fit the new dynamics embedded in the most recent process data acquired, and to compute the parameter λ that elucidates the magnitudes of the parametric drifts. Table 1 tabulates the estimated λ . Specifically, the absolute percentage estimation errors of the magnitude of parametric drifts λ_F and λ_{k_0} are 0.63%, and 1.25% for the first occurrence of process disturbances in the feed flow rate F , and frequency factor k_0 , respectively, demonstrating the

Table 1. Estimation of Parametric Drifts with the Proposed PIRNN-Enhanced Online Learning Mechanism^a

	$F(\text{m}^3/\text{h})$	$k_0(\text{m}^3/\text{kmol}\cdot\text{h})$
First disturbance	Actual = $1.60 \cdot F_{t=0}$ Estimated = $1.59 \cdot F_{t=0}$	Actual = $0.80 \cdot k_{0,t=0}$ Estimated = $0.79 \cdot k_{0,t=0}$
Absolute % error	0.63%	1.25%
Second disturbance	Actual = $2.30 \cdot F_{t=0}$ Estimated = $2.31 \cdot F_{t=0}$	Actual = $0.30 \cdot k_{0,t=0}$ Estimated = $0.29 \cdot k_{0,t=0}$
Absolute % error	0.43%	3.33%

^a $t = 0$ denotes the initial values of F and k_0 at the start of the reaction

feasibility of using PIRNN as a means to quantify process disturbances caused by parametric drifts.

Subsequently, the PIRNN model is updated with the governing equation of eq 14 comprising the revised parametric values of $\lambda_F \cdot F$ and $\lambda_{k_0} \cdot k_0$. In contrast to offline training, model update in an online learning-based MPC scheme should be completed within one sampling time to yield the best possible control performance. In this work, approximately 3000 collocation points consisting of the initial system state vector x_0 and manipulated input u which are uniformly sampled from the designated operating regions are used to update the PIRNN model to adapt to the new process dynamics engendered by the various disturbances. The total time taken for disturbance quantification and PIRNN model update is approximately 30 s on an NVIDIA RTX A5000 (24 GB) GPU, which is less than one sampling time $\Delta = 1 \times 10^{-2}$ hr = 36 s. As shown in Figure 4 and Figure 6 from the 15th to 19th sampling time, the PIRNN-enhanced online learning mechanism outperforms the traditional purely data-driven online update strategy in terms of the MSE error E_{RNN} and the control performance of the LMPC scheme. This demonstrates the effectiveness and the viability of exploiting PIRNN-enhanced update for boosting the efficacy of real-time process control.

Additionally, system disturbances of larger magnitudes are introduced at $t = 0.19$ hr, which similarly result in considerable deviations of the system states from their set points, as shown in Figure 4. This further manifests in the sudden increase in MSE errors $E_{RNN}(t)$ that transcend the error threshold E_T at the 20th sampling time (Figure 6). The onset of significant model–plant mismatch post $t = 0.19$ hr leads to the violation of threshold $E_{RNN}(t)$ for all of the following 5 sampling periods, and this finally triggers the second online update at $t = 0.24$ hr. The PIRNN-enhanced online update strategy again exhibits favorable control performance as compared to that of the conventional purely data-driven online update. As shown in Figure 4 and Figure 6, following the model update at $t = 0.24$ hr, the PIRNN-enhanced scheme demonstrates a superior control performance in terms of adhering to the stipulated set points and having a lower MSE error E_{RNN} . The result of parametric drift quantification is shown in Table 1, where the absolute percentage estimation errors of the magnitude of parametric drifts λ_F and λ_{k_0} are 0.43%, and 3.33% for the second occurrence of process disturbances in the feed flow rate F , and frequency factor k_0 , respectively. Moreover, it is noticed that the MSE error E_{RNN} post the second model update using the purely data-driven scheme still exceeds the error threshold E_T as shown in Figure 6 following the 24th sampling time. This is in stark contrast to the PIRNN-enhanced model update mechanism, where the MSE error E_{RNN} post the second model update is always below the error threshold E_T . This observation

underscores the inadequacy of a purely data-driven online update technique in mitigating large parametric drifts, as the data acquired may not be sufficient to update the model to a desired accuracy. The adoption of the PIRNN-enhanced model update mechanism is therefore advantageous to the conventional purely data-driven technique. Finally, it is also observed in Figure 5 that the manipulated inputs ultimately reach a new steady state, and this is a consequence of process parametric drift, where controlling and maintaining the manipulated inputs at their original steady-state set points no longer lead to the desired system state profiles.

CONCLUSION

This work presented the forward and inverse physics-informed recurrent neural network modeling approaches for modeling a nominal system and estimating uncertain parameters, respectively. The forward and inverse modeling approaches were then utilized to design a PIRNN-enhanced online learning strategy that aims to improve PIRNN models following the error-triggered update mechanism using the latest process data. The ML-based MPC was developed using the online updated PIRNN models to stabilize the nonlinear system. Finally, we compared the performance of the three PIRNN modeling approaches, including the proposed PIRNN-enhanced online learning scheme with estimation of uncertain parameters via the inverse problem technique, the PIRNN with standard purely data-driven online update, and the PIRNN without update, and demonstrated that the proposed integration of forward and inverse PIRNN modeling approach achieved the highest prediction accuracy and the best closed-loop performance within MPC.

APPENDIX

Description of CSTR and Process Parameters. The CSTR is assumed to be nonisothermal and is well-mixed. In addition, an irreversible second-order exothermic reaction, which proceeds to convert the reactant A to product B (i.e., $A \rightarrow B$), is considered to take place in the CSTR. The CSTR is furnished with a heating jacket that serves to regulate the rate of reaction by either removing or supplying heat energy at a rate of Q . The system dynamics can be collectively described by the following materials and energy balance equations of eq 14a, and eq 14b, respectively, which in turn underpin the development of physics-based learning algorithms.

$$\frac{dC_A}{dt} = \frac{F}{V}(C_{A0} - C_A) - k_0 e^{-E/RT} C_A^2 \quad (14a)$$

$$\frac{dT}{dt} = \frac{F}{V}(T_0 - T) + \frac{-\Delta H}{\rho_L C_p} k_0 e^{-E/RT} C_A^2 + \frac{Q}{\rho_L C_p V} \quad (14b)$$

where the inlet concentration of the reacting species A and the feed volumetric flow rate are represented by C_{A0} and F , respectively. The concentration of species A and the volume of the reacting fluid in the CSTR are denoted by C_A and V , respectively. The temperature dependence of the reaction rate is characterized by the Arrhenius equation $k_0 e^{-E/RT}$, with k_0 , E , and R denoting the frequency factor, activation energy, and the universal gas constant, respectively. Additionally, the temperatures of feed and reacting liquid in the CSTR are represented by T_0 and T , respectively. The enthalpy change of the reaction system is denoted by ΔH . Moreover, constant

density of ρ_L and heat capacity of C_p are assumed for the reacting liquid in the CSTR. Table 2 summarizes the numerical

Table 2. Summary of the Process Parameter Values Adopted in the Dynamic Model of CSTR

$F = 5 \text{ m}^3/\text{h}$	$V = 1 \text{ m}^3$
$k_0 = 8.46 \times 10^6 \text{ m}^3/\text{kmol}\cdot\text{h}$	$E = 5 \times 10^4 \text{ kJ/kmol}$
$R = 8.314 \text{ kJ/kmol}\cdot\text{K}$	$T_0 = 300 \text{ K}$
$\Delta H = -1.15 \times 10^4 \text{ kJ/kmol}$	$\rho_L = 1000 \text{ kg/m}^3$
$C_p = 0.231 \text{ kJ/kg}\cdot\text{K}$	$C_{A0s} = 4 \text{ kmol/m}^3$
$Q_c = 0.0 \text{ kJ/h}$	$C_{As} = 1.95 \text{ kmol/m}^3$
$T_s = 402 \text{ K}$	

values adopted for the various parameters in the governing equation of eq 14, and the respective steady-state values for the controlled (i.e., C_A and T) and manipulated (i.e., Q and C_{A0}) variables. In addition, the system states and the manipulated inputs are standardized to be expressed by their respective deviation variables: $\Delta C_A = C_A - C_{A_s}$, $\Delta T = T - T_s$, $\Delta C_{A0} = C_{A0} - C_{A0_s}$, and $\Delta Q = Q - Q_s$. Furthermore, physical limits are imposed on the manipulated inputs, where $|\Delta C_{A0}| \leq 3.5 \text{ kmol/m}^3$, and $|\Delta Q| \leq 5 \times 10^5 \text{ kJ/h}$. Collectively, the system states and the manipulated inputs of the closed-loop CSTR system are denoted by $x^T = [\Delta C_A, \Delta T]$, and $u^T = [\Delta C_{A0}, \Delta Q]$, respectively. Under these notations, the origin of the state-space thus represents an equilibrium point of the dynamic system (i.e., $(x_s^*, u_s^*) = (0, 0)$). The sampling time Δ , which is equal to the prediction horizon of the PIRNN model, is specified as 10^{-2} hr , and the nonlinear optimization problem of the LMPC of eq 13 is solved using PyIpopt, which is a Python interface to the state-of-the-art IPOPT solver package for solving nonlinear optimization problems.

Training process of the PIRNN model. The collocation points comprising the initial system states ΔC_A and ΔT , as shown in Figure 7, and the manipulated inputs ΔC_{A0} and ΔQ are

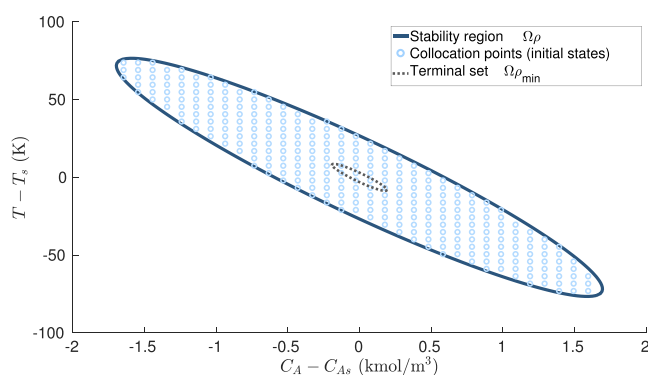


Figure 7. Distribution of collocation points adopted in the development of the forward problem of the PIRNN model, where the collocation points are sampled uniformly across the stability region Ω_p .

uniformly sampled across the closed-loop stability region Ω_p , and the physical limits of the manipulated variables (i.e., $-3.5 \text{ kmol/m}^3 \leq \Delta C_{A0} \leq 3.5 \text{ kmol/m}^3$ and $-5 \times 10^5 \text{ kJ/h} \leq \Delta Q \leq 5 \times 10^5 \text{ kJ/h}$) defined for the CSTR, respectively. Functionally, the collocation points are utilized to endow the PIRNN with a priori mechanistic knowledge by embedding the physical principles imposed by eq 14 into the network architecture. Altogether, 300 pairs of initial states are sampled across the

closed-loop stability region Ω_p , as shown in Figure 7, where each pair is then subject to 800 pairs of uniformly sampled manipulated inputs in a sample-and-hold fashion (i.e., the manipulated variables remain constant for one sampling period Δ , which also corresponds to one prediction horizon of the PIRNN model). The uniform sampling ultimately yields a total of 240,000 collocation points (i.e., 240,000 different combinations of 300 pairs of the system states ΔC_A and ΔT with 800 pairs of the manipulated inputs ΔC_{A0} and ΔQ) for training the PIRNN model. To assess model performance, the collocation points are further split into training (60%), validation (20%), and test (20%) data sets.

With the collocation data sampled, the PIRNN model is trained using the optimizer = *Adam*, learning rate = 0.001, weight initialization $\sim U(-\sqrt{k}, \sqrt{k})$ where $k = \frac{1}{\text{hidden layers size}}$, and activation function = *tanh*. Additionally, the early stopping technique is exploited to facilitate the training process by reducing overfitting without compromising on model accuracy. Specifically, an arbitrary large number of training epochs (e.g., 2000 in this work) is initially specified, and the training will be terminated immediately once the network weight updates no longer yield an improvement on the validation data set after a successive number of training iterations (e.g., 20 in this work).

Under the aforementioned configurations, the PIRNN model is constructed with 4 input features (i.e., $F_{nn}(\tilde{x}, u)$, with $\tilde{x} \in \mathbb{R}^2$ and $u \in \mathbb{R}^2$) using the state-of-the-art deep learning library PyTorch in the Python programming language. The PIRNN is developed to predict the evolution of the system states (ΔC_A and ΔT) for one sampling period $\Delta = 1 \times 10^{-2} \text{ hr}$ based on the initial system states ΔC_A and ΔT at $t = t_0$ hr, and the manipulated inputs ΔC_{A0} and ΔQ , which remain constant for a sampling period Δ . A time step of $h_c = 1 \times 10^{-3} \text{ hr}$ is chosen as the intermediate time step for the PIRNN model, which corresponds to the time interval between two consecutive internal states h_{t-1} and h_t of the unfolded PIRNN, as shown in Figure 1. Furthermore, all the dynamic system states that span from $t = t_0$ hr to $t = t_0 + \Delta$ hr at an interval of h_c are treated as the internal states of the PIRNN, which are output by the PIRNN model as the predicted system states for one sampling time forward. Note that the time step h_c adopted should be sufficiently small in order to warrant an approximation of the state derivatives (i.e., $\frac{dC_A}{dt}$ and $\frac{dT}{dt}$) to the desired accuracy using the finite difference method on the output of the PIRNN model. Moreover, to facilitate a more efficient learning process, all inputs to the PIRNN model are standardized by subtracting the arithmetic mean of the various input variables, and dividing by their respective standard deviations derived from the training data set.

AUTHOR INFORMATION

Corresponding Author

Zhe Wu – Department of Chemical and Biomolecular Engineering, National University of Singapore, 117585, Singapore; orcid.org/0000-0002-2923-149X; Email: wuzhe@nus.edu.sg

Author

Yingzhe Zheng – Department of Chemical and Biomolecular Engineering, National University of Singapore, 117585, Singapore

Complete contact information is available at:

<https://pubs.acs.org/10.1021/acs.iecr.2c03691>

Notes

The authors declare no competing financial interest.

ACKNOWLEDGMENTS

Financial support from the NUS Start-up grant (R-279-000-656-731) is gratefully acknowledged.

REFERENCES

- (1) Wu, Z.; Tran, A.; Rincon, D.; Christofides, P. D. Machine learning-based predictive control of nonlinear processes. Part I: theory. *AIChE J.* **2019**, *65*, No. e16729.
- (2) Zhao, T.; Zheng, Y.; Gong, J.; Wu, Z. Machine learning-based reduced-order modeling and predictive control of nonlinear processes. *Chem. Eng. Res. Des.* **2022**, *179*, 435–451.
- (3) Zheng, Y.; Wang, X.; Wu, Z. Machine Learning Modeling and Predictive Control of the Batch Crystallization Process. *Ind. Eng. Chem. Res.* **2022**, *61*, 5578–5592.
- (4) Chee, E.; Wong, W. C.; Wang, X. An integrated approach for machine-learning-based system identification of dynamical systems under control: application towards the model predictive control of a highly nonlinear reactor system. *Frontiers of Chemical Science and Engineering* **2022**, *16*, 237–250.
- (5) Ren, Y. M.; Alhajeri, M. S.; Luo, J.; Chen, S.; Abdullah, F.; Wu, Z.; Christofides, P. D. A tutorial review of neural network modeling approaches for model predictive control. *Comput. Chem. Eng.* **2022**, *165*, 107956.
- (6) Hu, C.; Cao, Y.; Wu, Z. Online Machine Learning Modeling and Predictive Control of Nonlinear Systems With Scheduled Mode Transitions. *AIChE J.* **2023**, *69*, No. e17882.
- (7) Raissi, M.; Perdikaris, P.; Karniadakis, G. E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* **2019**, *378*, 686–707.
- (8) Shah, P.; Sherif, M. Z.; Bangi, M. S. F.; Kravaris, C.; Kwon, J. S.-I.; Botre, C.; Hirota, J. Deep neural network-based hybrid modeling and experimental validation for an industry-scale fermentation process: Identification of time-varying dependencies among parameters. *Chemical Engineering Journal* **2022**, *441*, 135643.
- (9) Lee, D.; Jayaraman, A.; Kwon, J. S.-I. Development of a hybrid model for a partially known intracellular signaling pathway through correction term estimation and neural network modeling. *PLoS Computational Biology* **2020**, *16*, No. e1008472.
- (10) Wu, Z.; Rincon, D.; Christofides, P. D. Process structure-based recurrent neural network modeling for model predictive control of nonlinear processes. *Journal of Process Control* **2020**, *89*, 74–84.
- (11) Bhadriraju, B.; Bangi, M. S. F.; Narasingam, A.; Kwon, J. S.-I. Operable adaptive sparse identification of systems: Application to chemical processes. *AIChE J.* **2020**, *66*, No. e16980.
- (12) Karniadakis, G. E.; Kevrekidis, I. G.; Lu, L.; Perdikaris, P.; Wang, S.; Yang, L. Physics-informed machine learning. *Nature Reviews Physics* **2021**, *3*, 422–440.
- (13) Bangi, M. S. F.; Kao, K.; Kwon, J. S. Physics-informed neural networks for hybrid modeling of lab-scale batch fermentation for β -carotene production using *Saccharomyces cerevisiae*. *Chem. Eng. Res. Des.* **2022**, *179*, 415–423.
- (14) Alhajeri, M. S.; Abdullah, F.; Wu, Z.; Christofides, P. D. Physics-informed machine learning modeling for predictive control using noisy data. *Chem. Eng. Res. Des.* **2022**, *186*, 34–49.
- (15) Cai, S.; Wang, Z.; Wang, S.; Perdikaris, P.; Karniadakis, G. E. Physics-informed neural networks for heat transfer problems. *Journal of Heat Transfer* **2021**, *143*, 060801.
- (16) Sahli Costabal, F.; Yang, Y.; Perdikaris, P.; Hurtado, D. E.; Kuhl, E. Physics-informed neural networks for cardiac activation mapping. *Frontiers in Physics* **2020**, *8*, 42.
- (17) Bangi, M. S. F.; Kwon, J. S.-I. Deep hybrid modeling of chemical process: Application to hydraulic fracturing. *Comput. Chem. Eng.* **2020**, *134*, 106696.
- (18) Kimaev, G.; Ricardez-Sandoval, L. A. Nonlinear model predictive control of a multiscale thin film deposition process using artificial neural networks. *Chem. Eng. Sci.* **2019**, *207*, 1230–1245.
- (19) Kimaev, G.; Ricardez-Sandoval, L. A. Artificial neural network discrimination for parameter estimation and optimal product design of thin films manufactured by chemical vapor deposition. *J. Phys. Chem. C* **2020**, *124*, 18615–18627.
- (20) Mendiola-Rodriguez, T. A.; Ricardez-Sandoval, L. A. Robust control for anaerobic digestion systems of Tequila vinasses under uncertainty: A Deep Deterministic Policy Gradient Algorithm. *Digital Chemical Engineering* **2022**, *3*, 100023.
- (21) Zheng, Y.; Zhao, T.; Wang, X.; Wu, Z. Online learning-based predictive control of crystallization processes under batch-to-batch parametric drift. *AIChE J.* **2022**, *68*, No. e17815.
- (22) Zheng, Y.; Hu, C.; Wang, X.; Wu, Z. Physics-Informed Recurrent Neural Network Modeling for Predictive Control of Nonlinear Processes. *Journal of Process Control* **2022**.
- (23) Lin, Y.; Sontag, E. D. A universal formula for stabilization with bounded controls. *Systems & control letters* **1991**, *16*, 393–397.
- (24) Ying, X. An overview of overfitting and its solutions. *Journal of Physics: Conference Series* **2019**, *1168*, 022022.
- (25) Mayne, D. Q.; Seron, M. M.; Raković, S. V. Robust model predictive control of constrained linear systems with bounded disturbances. *Automatica* **2005**, *41*, 219–224.
- (26) Ricardez-Sandoval, L. A.; Budman, H. M.; Douglas, P. L. Simultaneous design and control of processes under uncertainty: A robust modelling approach. *Journal of Process Control* **2008**, *18*, 735–752.
- (27) Hu, C.; Jaimoukha, I. M. Robust H_2 and H_∞ state feedback control for discrete-time polytopic systems using an iterative LMI based procedure. 2020 European Control Conference (ECC). Saint Petersburg, Russia, 2020; pp 621–626.
- (28) Hu, C.; Jaimoukha, I. M. New iterative linear matrix inequality based procedure for H_2 and H_∞ state feedback control of continuous-time polytopic systems. *International Journal of Robust and Nonlinear Control* **2021**, *31*, 51–68.
- (29) Denis-Vidal, L.; Joly-Blanchard, G.; Noiret, C. Some effective approaches to check the identifiability of uncontrolled nonlinear systems. *Mathematics and computers in simulation* **2001**, *57*, 35–44.
- (30) Hengl, S.; Kreutz, C.; Timmer, J.; Maiwald, T. Data-based identifiability analysis of non-linear dynamical models. *bioinformatics* **2007**, *23*, 2612–2618.
- (31) Geffen, D.; Findeisen, R.; Schliemann, M.; Allgower, F.; Guay, M. Observability based parameter identifiability for biochemical reaction networks. 2008 American Control Conference **2008**, 2130–2135.
- (32) Valipour, M.; Ricardez-Sandoval, L. A. Abridged Gaussian sum extended Kalman filter for nonlinear state estimation under non-Gaussian process uncertainties. *Comput. Chem. Eng.* **2021**, *155*, 107534.
- (33) Valipour, M.; Ricardez-Sandoval, L. A. Constrained abridged Gaussian sum extended Kalman filter: constrained nonlinear systems with non-Gaussian noises and uncertainties. *Ind. Eng. Chem. Res.* **2021**, *60*, 17110–17127.
- (34) Paszke, A.; Gross, S.; Massa, F.; Lerer, A.; Bradbury, J.; Chanan, G.; Killeen, T.; Lin, Z.; Gimelshein, N.; Antiga, L.; et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems* **2019**, *32*, 8026–8037.
- (35) Wu, Z.; Rincon, D.; Gu, Q.; Christofides, P. D. Statistical Machine Learning in Model Predictive Control of Nonlinear Processes. *Mathematics* **2021**, *9*, 1912.
- (36) Wu, Z.; Alnajdi, A.; Gu, Q.; Christofides, P. D. Statistical machine-learning-based predictive control of uncertain nonlinear processes. *AIChE J.* **2022**, *68*, No. e17642.
- (37) Hoi, S. C. H.; Sahoo, D.; Lu, J.; Zhao, P. Online learning: A comprehensive survey. *Neurocomputing* **2021**, *459*, 249–289.

(38) Wu, Z.; Rincon, D.; Christofides, P. D. Real-time adaptive machine-learning-based predictive control of nonlinear processes. *Ind. Eng. Chem. Res.* **2020**, *59*, 2275–2290.